Review 2 Agent 13 Secure Shredding Agent

Chapter 3: System Architecture & Design

- **3.1 High-Level Architecture**
    - Agent 13 has been implemented as an independent background service for Phase A. This review describes the architecture, component responsibilities, safety controls and development method. The complete CLI-to-C workflow functions without a master shell, LLM or MCP server.
    - Current execution path
      User -> agent13ctl CLI -> Unix socket with JSON
      -> Python service -> Validation and confirmation checks
      -> ctypes wrapper -> C shared library
      -> Linux system calls -> Allowed filesystem directory
    - The CLI is a client process. The service remains available between requests, waits on a selector and accepts local socket connections. The service calls native functions through ctypes in the same service process; the shared library is not another daemon. JSON results travel back through the socket to the CLI.
    - Future integration path not implemented
      Future Master Shell -> Shared Agent Interface
      -> Agent 13 Service -> C Backend -> Linux Kernel
    - The future master would call the same public service API. Any natural-language interpretation or MCP translation would occur outside the current execution path. This preserves the working implementation and its validation when the class later agrees on a common interface.
- **3.2 Component Design**
    - Interface layer: bin/agent13ctl and agent13/cli.py parse health, info, inspect, preview and shred commands. agent13/client.py sends one versioned JSON request and validates the reply. Running shred without a confirmation token performs a preview, so the CLI defaults to dry-run behavior.
    - Service layer: agent13/service.py accepts requests and checks Linux SO_PEERCRED credentials. agent13/protocol.py validates action names, parameters, version and request ID. The Agent object manages confirmation tokens and dispatches fixed operations. There is no custom REPL, LangChain orchestration or intent-routing model in Phase A.
    - Native tooling layer: agent13/native.py maps the C ABI through ctypes, holds the allowed-root descriptor and converts native errors into structured results. include/agent13_shred.h declares agent13_inspect_at and agent13_shred_at; src/c/agent13_shred.c implements the Linux file operations.

- Lifecycle and integration: systemd/agent13.service provides user-service process management. A root lock and a socket-directory lock prevent cooperating duplicate instances. manifests/agent13.tools.json describes four proposed tools for future function calling; it is metadata, not an implemented MCP server.

- **3.3 Security & Guardrail Framework**
  - Confirmation workflow: inspect and preview do not overwrite or remove the target. Preview stores its fingerprint and returns a random token with a default 60-second lifetime. Execution consumes that known token before checking expiry, path matching and the native operation. Reuse, file replacement, mismatch or restart requires a fresh preview.
  - The fingerprint includes device, inode, size, nanosecond mtime and ctime, mode, UID and link count. C compares the opened file with the preview fingerprint and obtains an exclusive nonblocking advisory lock. Tokens reduce accidental execution and replay; they do not prove human intent against a malicious process running under the same UID.
  - Confinement: the allowed root and socket parent must be owned mode-0700 directories. The socket is mode 0600, and peers must have the service UID. Descriptor-relative traversal with O_NOFOLLOW rejects symlinks; dot components, traversal and reserved names are forbidden. Targets must be owned regular files with one link and suitable permissions.
  - Safe mutation: C creates a private staging directory, uses renameat2 with RENAME_NOREPLACE, and verifies that the staged name still identifies the opened file. It overwrites through the held descriptor, fsyncs, then unlinks the private staged name. A new file created at the original name is preserved. File and directory synchronization are separate operations [1, 2].
  - Failure handling: short writes and interrupted calls are handled explicitly. On failure, C attempts a no-clobber restore; otherwise it returns a recovery path. The error also reports bytes_written and unlinked, because failure can occur after partial overwrite or deletion. There is no automatic destructive retry.
  - Request policy uses an allowlist of fixed actions and strict JSON validation. No arbitrary shell command is accepted, so safety does not rely on a regex blacklist. Requests are bounded at 16 KiB, live previews at 256 and preview size at 64 MiB by default. Physical erasure, hostile same-UID processes and uncooperative writers remain documented limitations.

Chapter 4: Implementation Methodology

- **4.1 Development Environment & Tech Stack**
    - Platform and languages: the project targets Ubuntu/Debian Linux, Python 3.10 or later and C11 with Linux/POSIX APIs. The recorded verification environment was Ubuntu 22.04.5 on x86-64 with Python 3.10, GCC 11.4 and Make 4.3. The native library is compiled with warnings treated as errors.
    - Libraries and IPC: Python uses only its standard library, including ctypes, socket, json, selectors, secrets, logging and unittest. C uses openat, fstat/fstatat, flock, getrandom, renameat2, write, fsync and unlinkat. The build produces build/libagent13_shred.so; no cloud account, API key, LLM provider or MCP SDK is required.
    - Service management: the supplied systemd user unit runs the service without root privileges, creates its private runtime directory and sends logs to the journal. It supports normal start, stop, restart, status and enable/disable operations after manual installation. The unit was validated; live host installation and enablement were intentionally not performed.
- **4.2 Step-by-Step Implementation**
    - Step 1: Repository inspection and planning. The empty project directory was inspected first. docs/ROADMAP.md defined requirements, milestones, security checks, testing and future integration. docs/ARCHITECTURE.md then documented six diagrams before application code was written.
    - Step 2: C backend and native verification. A public C header and implementation were added, then compiled as a shared library. Tests exercised real overwrite and unlink behavior, short and interrupted I/O, random-generation errors, synchronization failures, file replacement during staging and recovery conflicts. All targets were disposable files.
    - Step 3: Python-to-C interface. The ctypes wrapper mapped function signatures and fixed-width structures, checked ABI version 1 and converted native status into exceptions and JSON-safe details. Wrapper tests verified inspection, deletion, changed files, permissions and unsupported targets before service development.
    - Step 4: Background service. A Unix socket server and versioned protocol were implemented with peer checks, resource limits, a monotonic token expiry clock and graceful signal handling. A real socket health request and clean shutdown were verified before continuing to the CLI.

- Step 5: CLI and process configuration. The required commands were implemented as socket clients, keeping all shredding logic in the service and C library. A systemd user unit and configuration guide were added. The service rejects malformed JSON, unknown actions and unexpected parameters; no prompt engineering or model-based tool matching is required.
- Step 6: Public integration interfaces. The provisional version 1.0 contract includes version, agent_id, request_id, action and parameters. Responses return success, data or error. The four manifest actions are health, inspect_target, preview_shred and execute_shred. The class-wide schema and any future MCP adapter remain unimplemented.
- Step 7: Comprehensive verification. The native fault-injection harness and 28 Python unittest tests passed. Coverage includes compilation, ctypes, health, socket IPC, dry-run behavior, actual disposable-file shredding, permissions, traversal, symlinks, hardlinks, token errors, file changes, JSON validation, duplicate services and graceful shutdown/restart.
- Step 8: Independent demonstration and documentation. The background-service demo inspected a 4,000-byte sample, preserved it in dry-run, confirmed overwrite and removal, rejected token reuse, displayed logs, and stopped/restarted cleanly. README, API, architecture, integration, testing, security and demo/viva guides were completed.
- Verification limits: systemd-analyze user-unit validation passed, while runtime lifecycle tests used directly launched service processes. Foreign-UID policy was unit-tested with controlled credentials rather than a newly created user account. Tests demonstrate logical behavior and do not certify physical storage erasure. The full evidence is recorded in docs/TESTING.md.

References

- [1] Linux man-pages. fsync 2. https://man7.org/linux/man-pages/man2/fsync.2.html
- [2] Linux man-pages. rename 2. https://man7.org/linux/man-pages/man2/rename.2.html
- Project evidence: docs/ARCHITECTURE.md, docs/API.md, docs/SECURITY.md, docs/TESTING.md and docs/PROGRESS.md; include/agent13_shred.h; src/c/agent13_shred.c; agent13/service.py; systemd/agent13.service.